

```
for i in range(1,gamesize):
    for j in range(1,gamesize + 1):
        allpossible.append([(i,j),(i + 1,j)])

for move in allpossible:
    if move in game:
        allpossible.remove(move)
```

Next, we'll generate a complete game tree that extends to a depth of three levels:

```
thetree = generate_tree(allpossible,0,3,game)
```

Now that we have our game tree, we can call our `minimax()` function:

```
move,score = minimax(1,thetree)
```

And finally, we can check the best move as follows:

```
print(thetree[move][0])
```

We see that the best move is [(4, 4), (4, 3)], the move that completes a square and earns us a point. Our AI can play dots and boxes, and choose the best moves! You can try other game board sizes, or different game scenarios, or different tree depths, and check whether our implementation of the minimax algorithm is able to perform well. In a sequel to this book, we'll discuss how to ensure that your AI doesn't become simultaneously self-aware and evil and decide to overthrow humanity.

Adding Enhancements

Now that you can perform minimax, you can use it for any game you happen to be playing. Or you can apply it to life decisions, thinking through the future and maximizing every minimum possibility. (The structure of the minimax algorithm will be the same for any competitive scenario, but in order to use our minimax code for a different game, we would have to write new code for the generation of the game tree, the enumeration of every possible move, and the calculation of game scores.)

The AI we've built here has very modest capabilities. It's only able to play one game, with one simple version of the rules. Depending on what